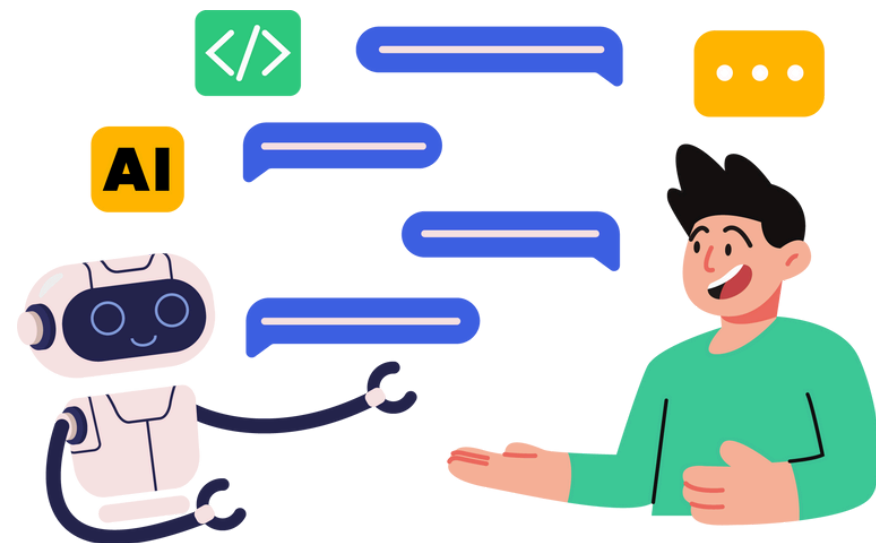
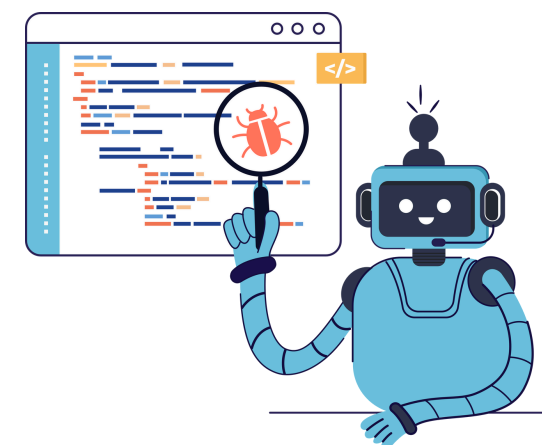
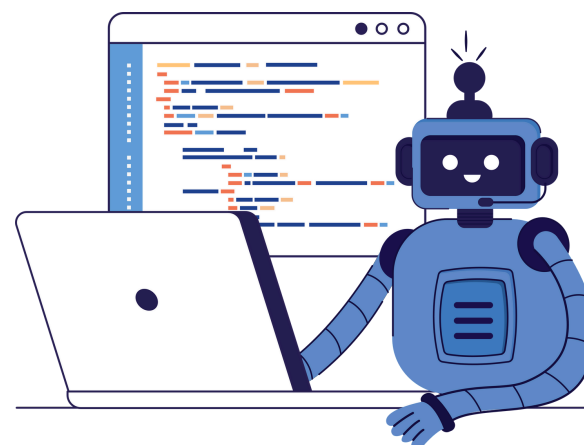
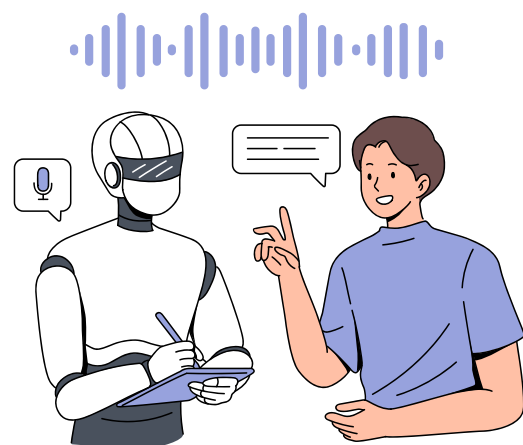
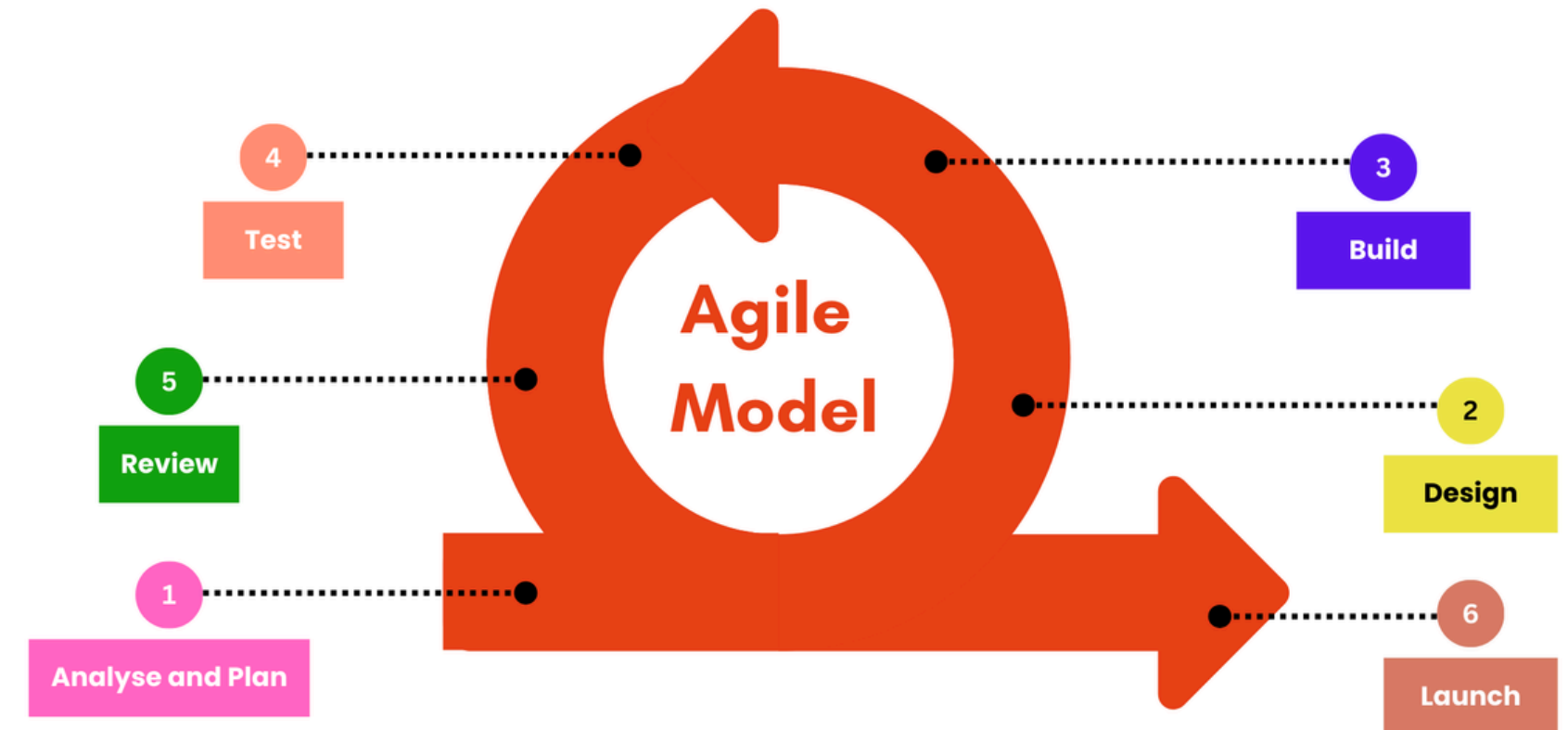
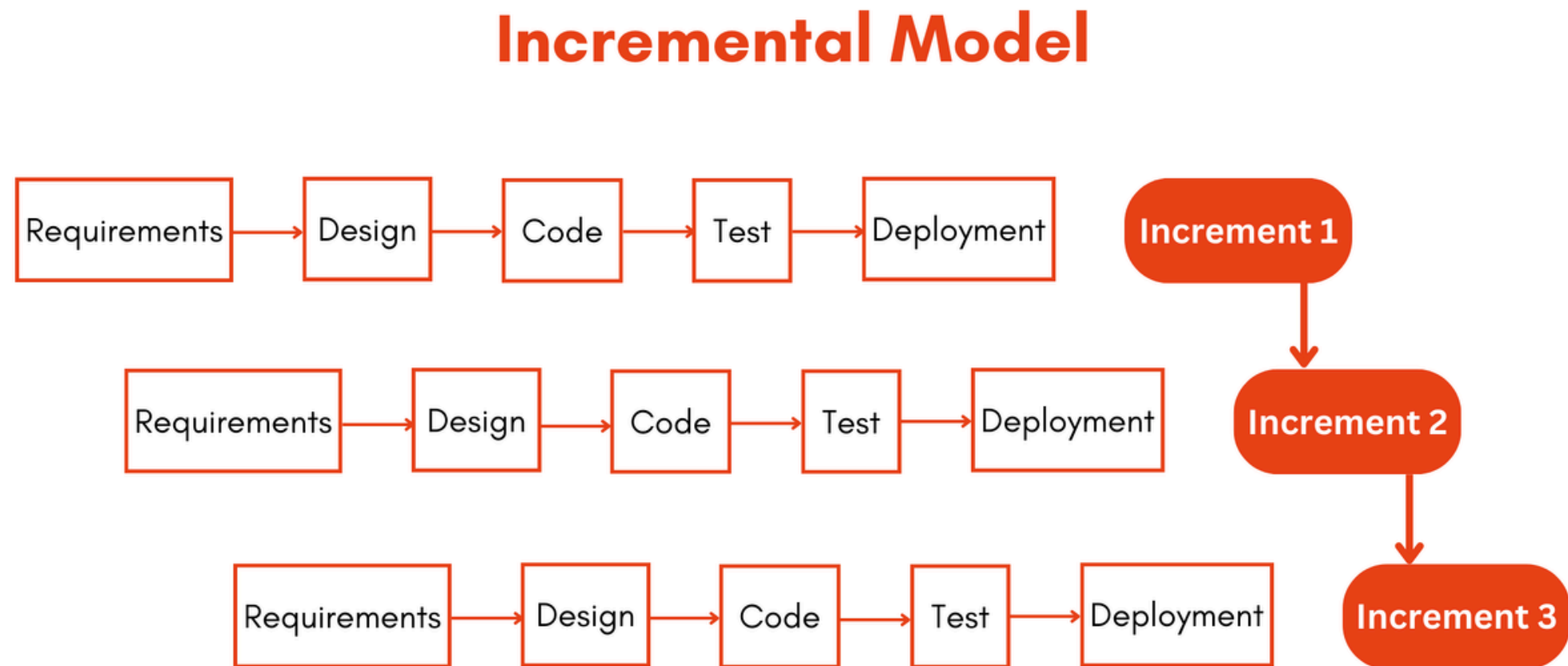
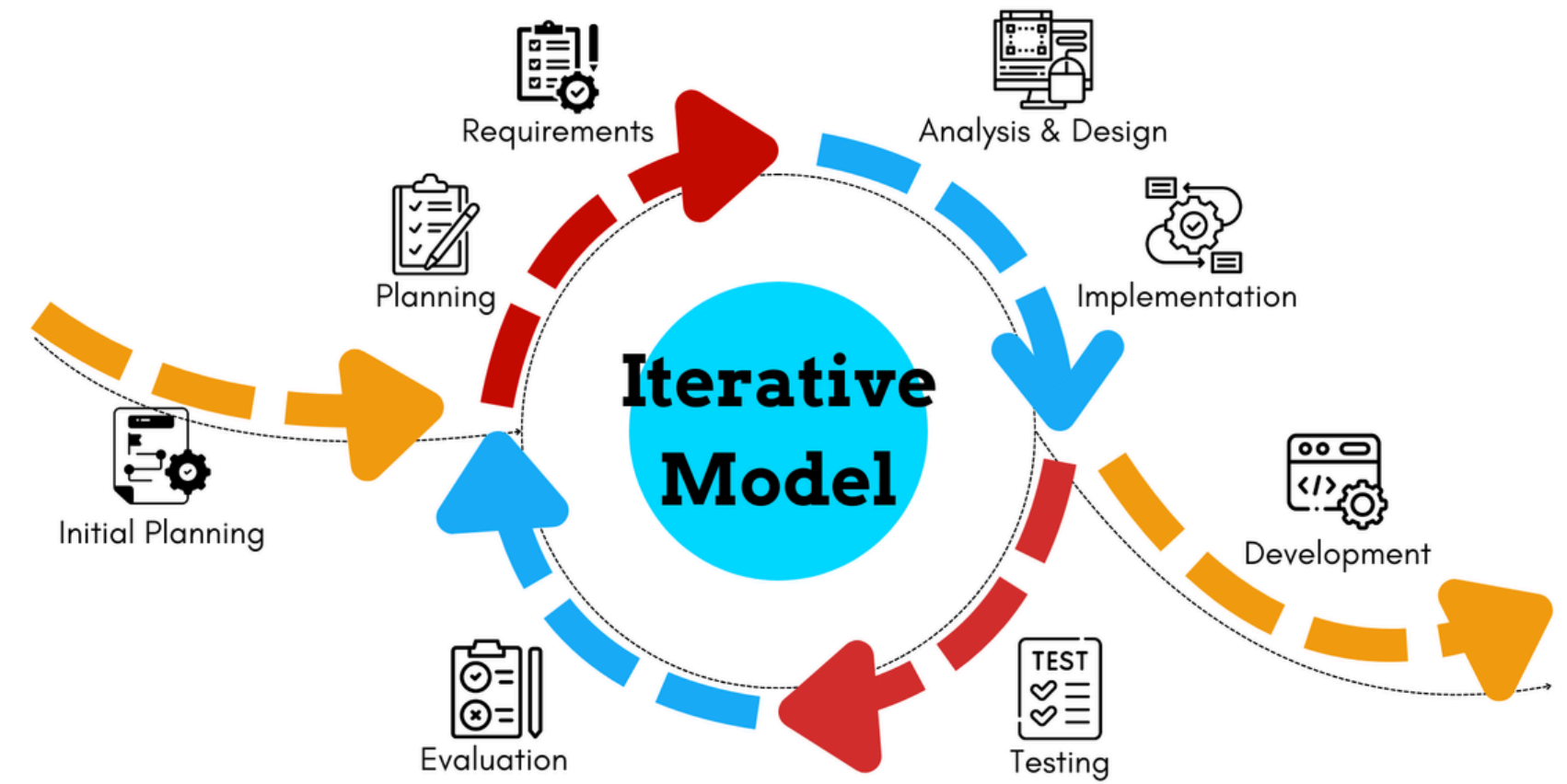
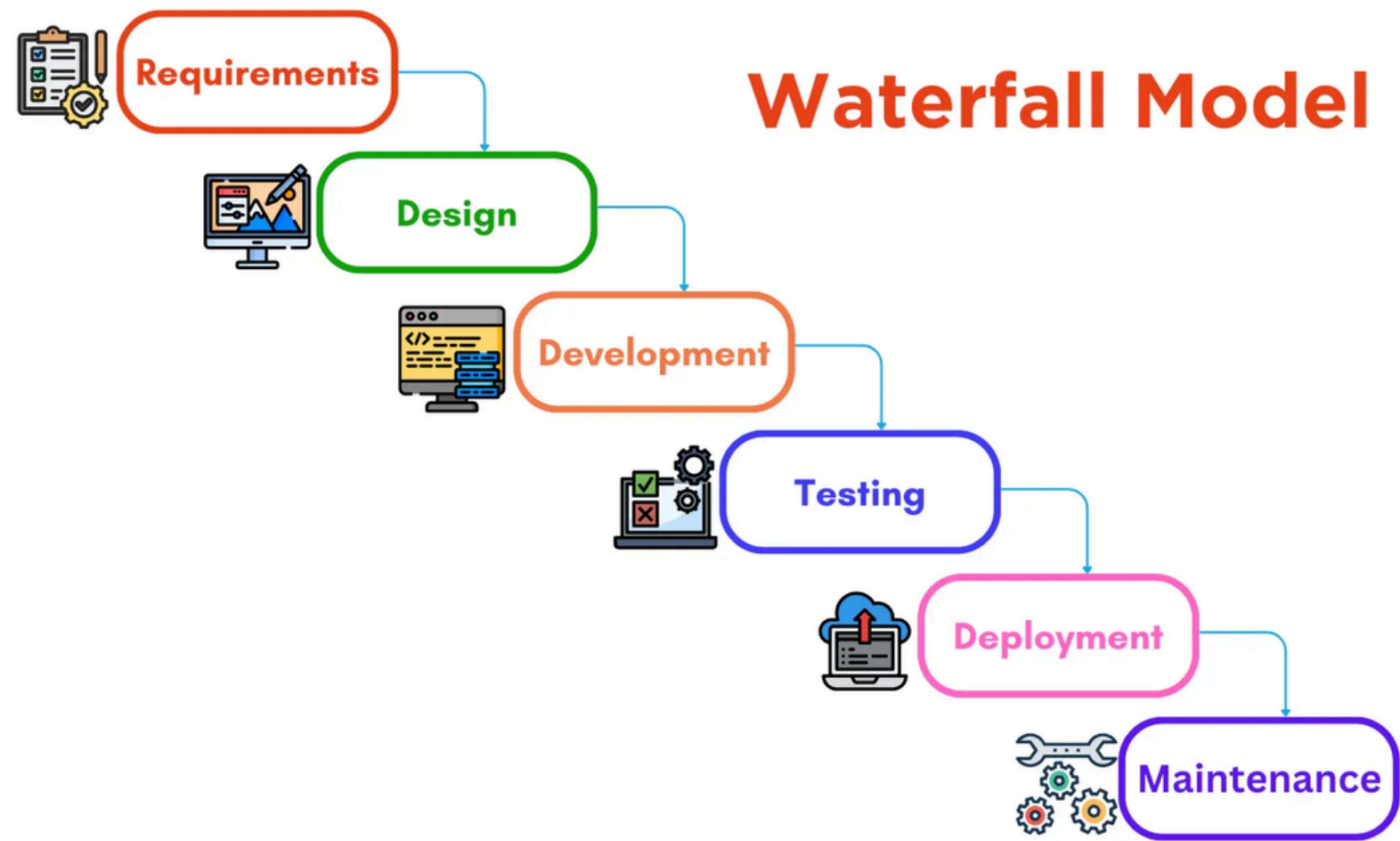




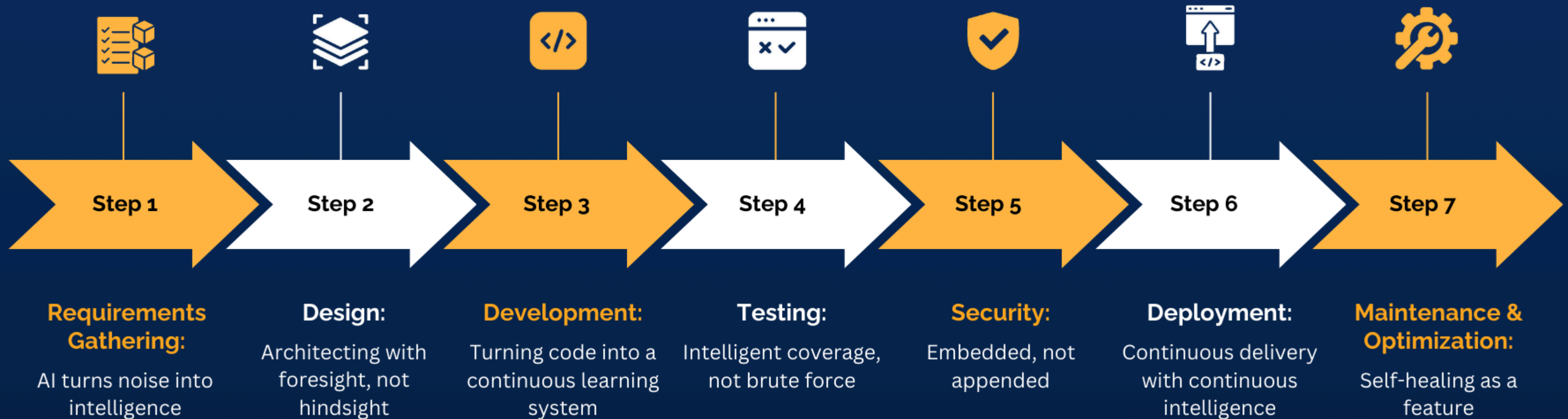
Overview of AI's role SDLC



सी डैक
CDAC



How AI is Transforming the Software Development Lifecycle (SDLC)

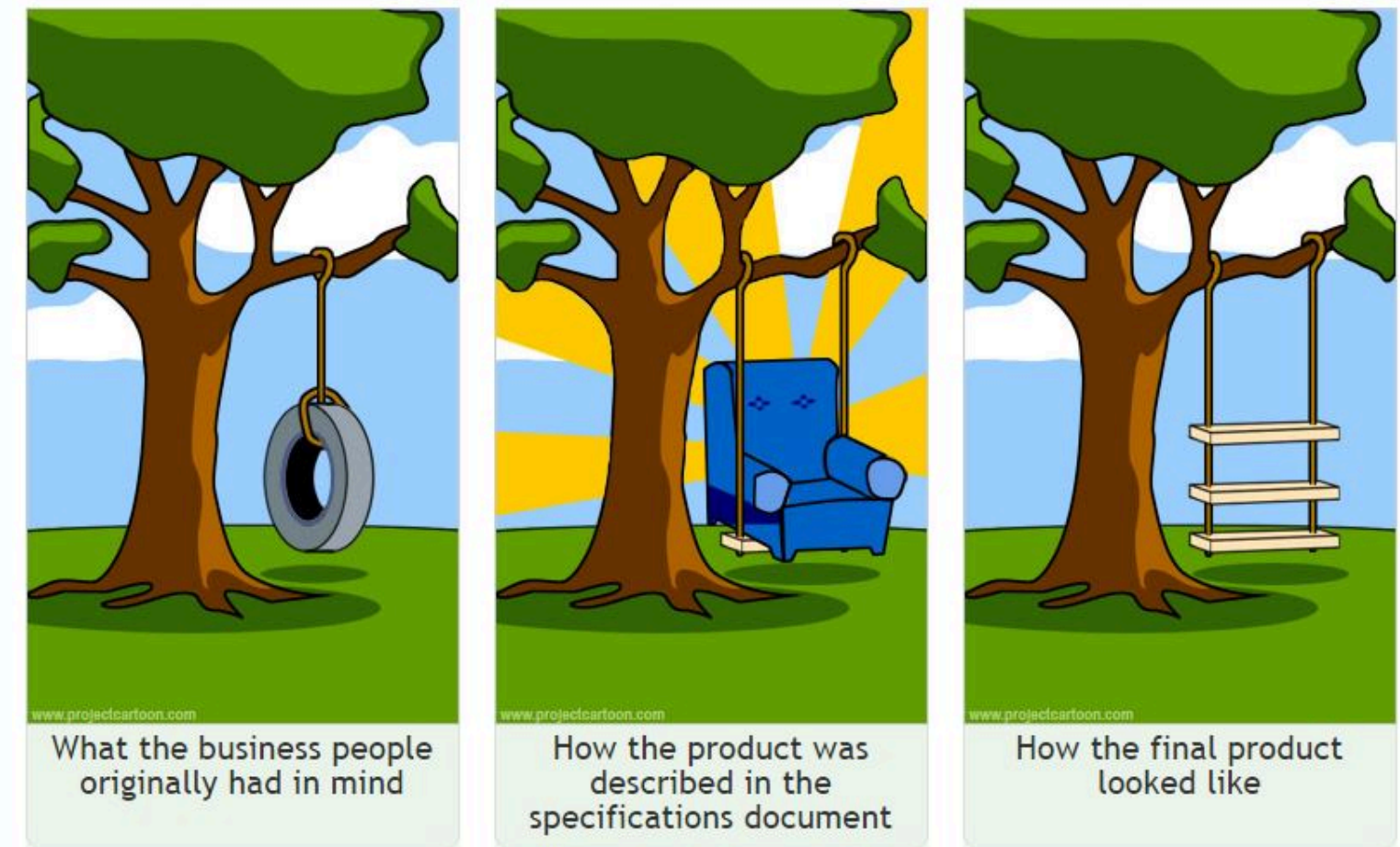


Requirements Gathering: AI turns noise into intelligence

Traditionally, requirement gathering relies heavily on manual interviews, stakeholder meetings, and inferred customer needs, leaving room for subjectivity and gaps.

With AI:

- NLP models mine product reviews, customer support chats, and usage logs to extract pain points and unmet needs.
- LLMs convert high-level business goals into user stories, flagging inconsistencies and missing logic



Why it matters:

- Product managers no longer have to guess what to build - they can mine it directly from user behavior
- AI makes early-stage planning scalable

Design: Architecting with foresight, not hindsight

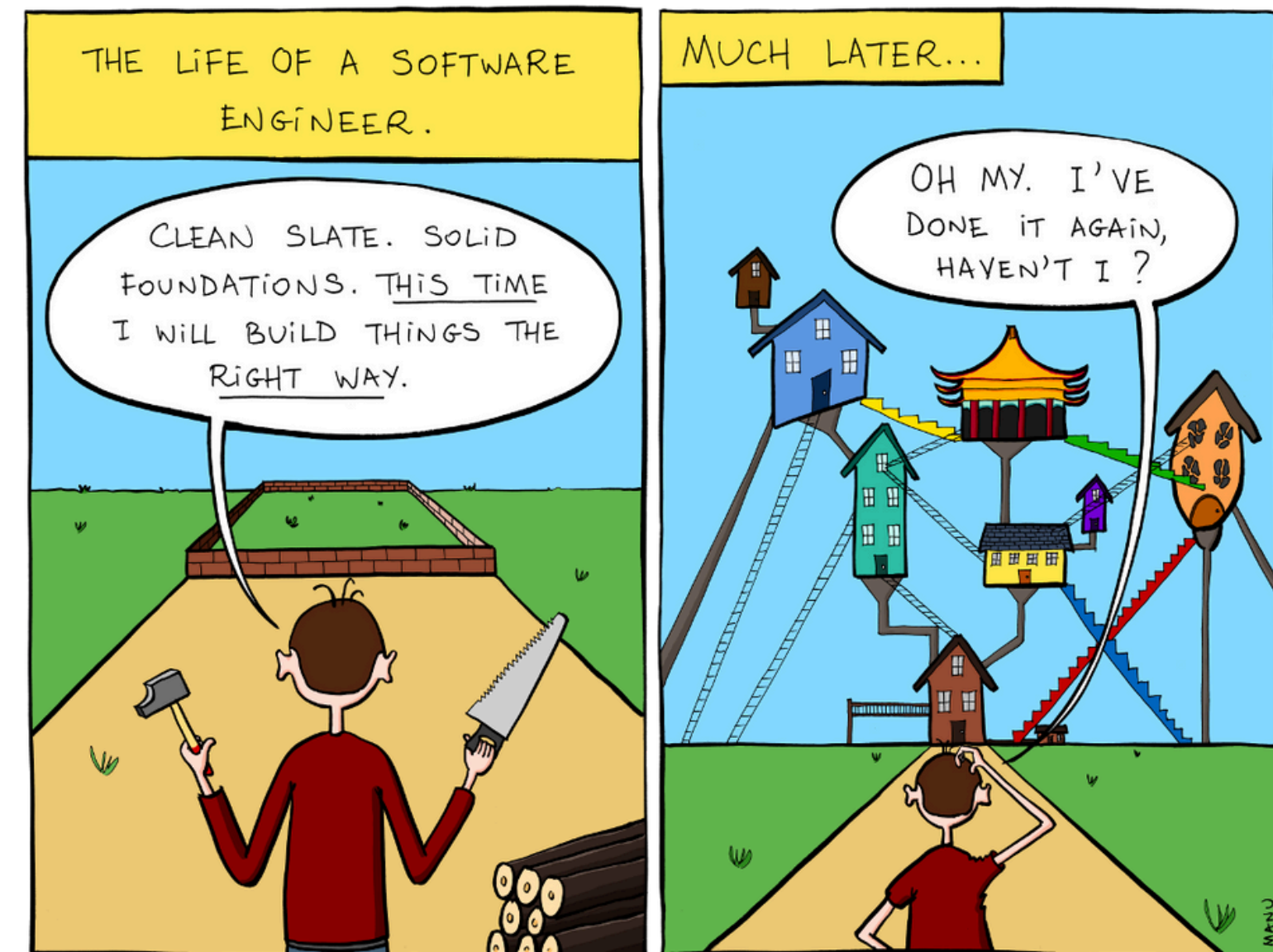
Architectural decisions are often driven by experience and intuition, but they rarely account for dynamic scaling, system degradation, or failure patterns until it's too late.

With AI:

- Tools simulate how proposed architectures respond to traffic, latency, and failure conditions.
- AI flags anti-patterns and suggests architectural refinements before a single line of code is written.

Why it matters:

- You go from “best practices” to data-backed designs. Every architectural choice is validated against real-world conditions before it's tested in production.



Development: Turning code into a continuous learning system

Development teams spend a significant portion of time on repetitive patterns like glue code, error handling, and scaffolding.

With AI:

- Copilots predict the developer's next move based on the function, file, and even team history.
- Code refactoring and optimization are continuous, not one-off activities.

Why it matters:

- Developers move from code producers to system thinkers. The AI captures tribal knowledge and makes engineering quality scalable.

Days before OpenAI



Days after OpenAI



Testing: Intelligent coverage, not brute force

Manual test writing is time-intensive and often fails to account for edge cases or systemic risk.

With AI:

- GenAI can generate test cases from requirements or existing code logic.
- Predictive models recommend which tests to run based on recent code changes and risk profiles.

Why it matters:

- Testing becomes adaptive and aligned to risk, change, and business criticality.

**WHEN IT'S UP TO
YOU TO FIX OLD
CODE WRITTEN BY OTHERS**



Security: Embedded, not appended

Security has traditionally been a late-stage checkpoint, leading to high-cost fixes and compliance gaps.

With AI:

- Tools like DeepCode or Snyk scan for vulnerabilities in real time.
- AI can simulate attacks based on threat patterns and prioritize remediation.

Why it matters:

- Security becomes a developer ally, not a blocker. Teams get continuous visibility without sacrificing velocity.

Deployment: Continuous delivery with continuous intelligence

Even with mature CI/CD, rollouts can be risky. AI mitigates this by understanding past deployments, rollback scenarios, and environment behavior.

With AI:

- Systems choose the optimal release strategy based on historical signals.
- Incident data trains models to avoid repeat failures.

Why it matters:

- Deployment becomes a controlled experiment—measured, reversible, and safe.

Maintenance & Optimization: Self-healing as a feature

Traditional incident response depends on alerts, dashboards, and war rooms.

With AI:

- Anomalies are flagged before users notice them
- AI correlates signals across logs, metrics, and traces to pinpoint root causes—and even resolve them automatically

Why it matters:

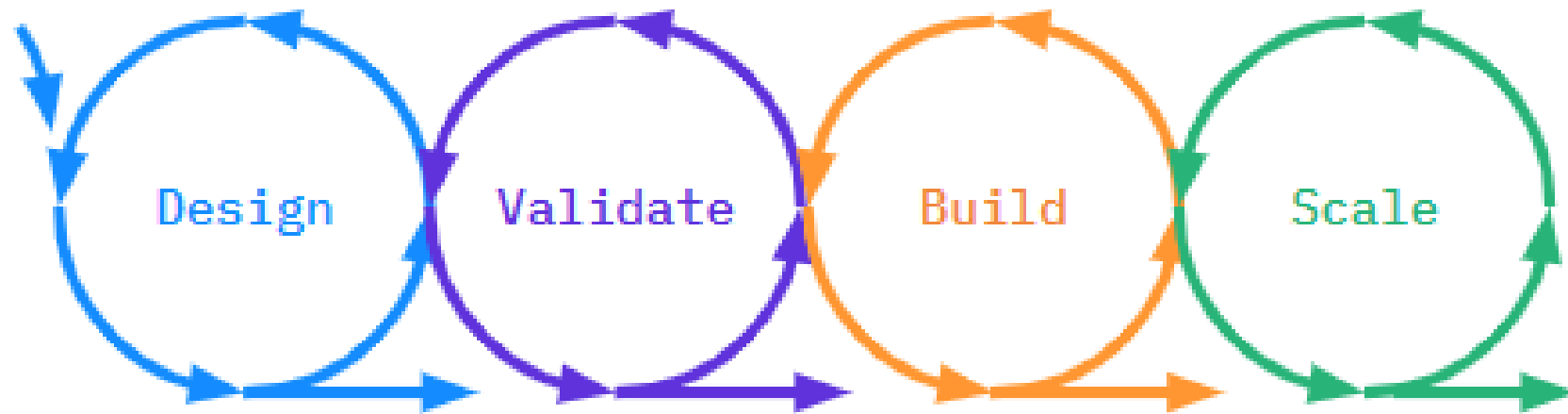
- Uptime isn't just protected; it's proactively engineered
- Engineers shift from firefighting to foresight

Then vs Now: AI in the SDLC

Phase	Then (Manual)	Now (AI-Driven)
Requirements	Stakeholder interviews	NLP mining of chat logs, usage data
Design	Static architecture diagrams	Simulated, stress-tested blueprints
Development	Manual coding	Copilot-assisted logic generation
Testing	Regression suites	GenAI-generated, risk-prioritized test cases
Deployment	Scripted CI/CD	AI-triggered rollouts, auto-rollbacks
Maintenance	Alerts and dashboards	Predictive diagnostics, self-healing ops

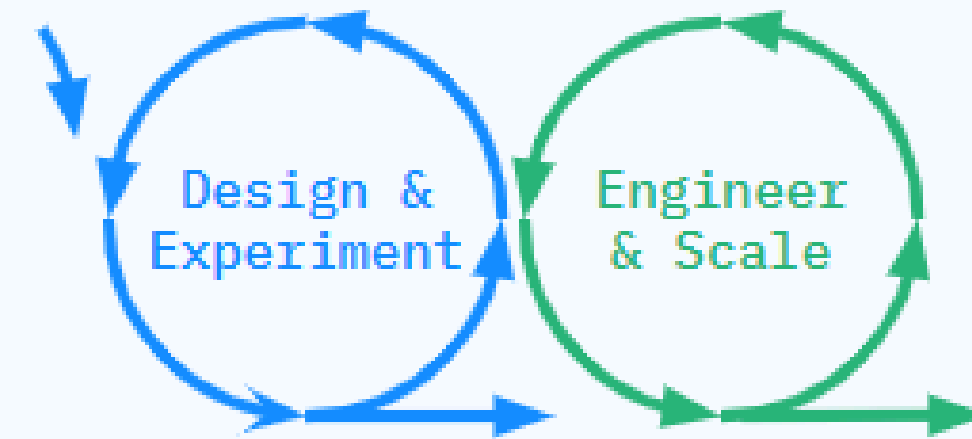
Software Development Life Cycle^(SDLC) compressed

BEFORE



- Fragmented SDLC Ownership;
- Scaling Challenges;
- Siloed Teams Increasing friction.

NOW



- Democratized development;
- Bridge between business and IT;
- Composability for business agility.

AI-native SDLC drives faster, customer-centric, and higher-quality innovation.

What AI Means for Software Engineers?

- **More time solving hard problems:** Repetitive code is delegated to copilots. Engineers focus on architecture, modeling, and outcomes
- **Human-AI pair programming:** Instead of solo coding, engineers now prompt, steer, and refactor AI-generated suggestions
- **New accountability layers:** Engineers must now validate, contextualize, and interpret what AI suggests and not blindly adopt it

Base

Foundations from C-DAC Courses

Add-On

AI Tools for SDLC



Make your own list of AI Tools to use in every phase of SDLC

Thank You